



TON Blockchain Jetton Standard Implementation Security Audit

TONTECH VERIFIED ON 27 MAR 2025

01 Summary

This audit report focuses on compliance with the [TEP-0074](#) Jetton Standard for the standard jetton implementation contracts. The implementation also follows the [TEP-0089](#) Discoverable Jetton Wallets standard, which extends [TEP-0074](#) with wallet discovery functionality.

Total findings	Resolved	Acknowledged	Declined	Unresolved
7	7	0	0	0

The findings were classified by the severity levels as following:

Critical	Major	Medium	Minor	Informational
0	0	0	0	7

02 Severity levels

Critical

These are the most severe security flaws in a smart contract. They can lead to immediate and significant risks, allowing attackers to gain unauthorized access,

bypass security mechanisms, or cause significant financial losses. Critical vulnerabilities require immediate attention and remediation.

Major

These vulnerabilities are significant but may not pose an immediate threat or have a lower probability of exploitation compared to critical vulnerabilities. However, they still have the potential to result in significant damage or security breaches if left unaddressed. They should be resolved in a timely manner to ensure the overall security of the smart contract.

Medium

Medium vulnerabilities indicate security weaknesses that may not have a high likelihood of exploitation or immediate serious consequences. However, they can still impact the overall security posture of the smart contract and should be fixed to enhance its resilience against potential attacks or vulnerabilities.

Minor

These vulnerabilities are less critical in nature and generally have a limited impact on the security of the smart contract. While they may not pose an immediate threat, they should not be overlooked, as they can compound with other vulnerabilities or serve as a stepping stone for potential attackers. Addressing them is important to maintain a robust security posture.

Informational

Suggestions for code style, architecture, and optimization to improve quality and efficiency. Not security risks, but enhance resilience.

03 Codebase

[Repository](#) ([ton-blockchain/token-contract](#))

The following commit contains code that was audited:

[Target commit](#) ([21e7844fa6dbed34e0f4c70eb5f0824409640a30](#))

Code and contract hashes of the commit are provided in the next section.

04 Scope

Source code files (inside of `ft/` directory):

File paths	Rollup SHA-256 hash of the files
<code>ft/discovery-params.fc</code>	1b9ebd482fc97725a28a5d301d9d2872e1d0dfeec8906d56ab62aa7fe45dace7
<code>ft/jetton-discovery.fc</code>	ce931d84dd416d8b2e2c1d1a98c633d747db2e1056dd78d76b39650176a5196e
<code>ft/jetton-minter-ICO.fc</code>	b428fbcc3e6a1056d50b0aeabe6c1cccd265aef0b0218a354b9783503891f703
<code>ft/jetton-minter-discoverable.fc</code>	0cdf38d924b3b3630f6fc923f2ba82651d0354f33b9f41e9afe1c955672bb56b
<code>ft/jetton-minter.fc</code>	4fc34d4a0dd8cd8453dcb3d5f1ff261346713792a6cab23783aecbef1a196530
<code>ft/jetton-utils.fc</code>	f825c69d4660f04267b2b0f2372d65819ac9c90b7bf828910bfc16a1bdcd9b19
<code>ft/jetton-wallet.fc</code>	9369b73c1938584ea65061f0d3a13043d2c32b07c8e217576fdbbfff953057a22
<code>ft/op-codes.fc</code>	803967bb1ab82be89ca4d9e530cd5f8b347426ab3c101c24c7377d09cfd543e0
<code>ft/params.fc</code>	65925e6e45e20dbc836dbec2dfd660e874aa7f1049d187ae543b469018de9a7a

Rollup is obtained by calling `sha256sum ft/*.fc`

Resulting hashes of the compiled smart contracts:

SC compile definition name	Compiled code cell hash
JettonMinter	0571976c63ec1b7550230a2609dbedb3 6e1b64ef8d022a16b34ea57063185b2f
JettonWallet	a760d629d5343e76d045017d9dc216fc 8a307a8377815feb2b0a5c490e733486

`wrappers/*.compile.ts` definitions were used for compiling contracts and getting their hashes.

05 Standard Compliance Assessment

The [TEP-0074](#) standard defines requirements for Jettons on TON Blockchain, while [TEP-0089](#) extends it with Wallet Discovery capabilities. Our assessment confirms that the implementation correctly follows all requirements of both standards. The implementation includes all required message handlers, get-methods, and follows the specified data structures and message formats.

Transfer Operation Implementation

Category	Severity	Location	Status
Compliance	Informational	ft/jetton-wallet.fc	Verified

Description

The `transfer#f8a7ea5` message handler implements the jetton transfer mechanism defined in [TEP-0074](#). This operation allows jetton owners to send jettons to other addresses and forms the primary jetton movement capability.

Implementation Details

The implementation follows the exact TL-B schema from the standard:

```
transfer#0f8a7ea5 query_id:uint64 amount:(VarUInteger 16) destination:MsgAddress
response_destination:MsgAddress custom_payload:(Maybe ^Cell)
forward_ton_amount:(VarUInteger 16)
forward_payload:(Either Cell ^Cell)
= InternalMsgBody;
```

The `transfer` operation correctly handles all required parameters:

- `query_id (uint64)` - arbitrary request number
- `amount (VarUInteger 16)` - jetton amount to transfer
- `destination (MsgAddress)` - destination address
- `response_destination (MsgAddress)` - where to send operation excess
- `custom_payload (Maybe ^Cell)` - optional custom data

6. `forward_ton_amount (VarUInteger 16)` - ton amount of the `transfer_notification` message to recipient
7. `forward_payload (Either Cell ^Cell)` - data to be sent to the destination

The implementation validates sender ownership, verifies sufficient balance, ensures adequate TON for transaction processing, and correctly calculates gas requirements. Upon validation, it decreases the sender's balance and creates a message to the recipient's wallet with proper state initialization.

Compliance Note

The implementation enforces all security requirements from [TEP-0074](#), including ownership verification and balance validation. The operation maintains consistent jetton supply by ensuring jettons deducted from the sender appear in the recipient's wallet. The bounce handling mechanism protects against jetton loss during failed transfers. This operation enables the jetton's primary utility — movement between addresses — while preserving the jetton's supply integrity.

Burn Operation Implementation

Category	Severity	Location	Status
Compliance	Informational	ft/jetton-wallet.fc	Verified

Description

The `burn#595f07bc` message handler implements the jetton supply reduction mechanism defined in [TEP-0074](#). This operation permanently removes jettons from circulation by reducing both the wallet balance and the total supply.

Implementation Details

The implementation follows the exact TL-B schema from the standard:

```
burn#595f07bc query_id:uint64 amount:(VarUInteger 16)
               response_destination:MsgAddress custom_payload:(Maybe ^Cell)
               = InternalMsgBody;
```

The `burn` operation correctly processes all required parameters:

- 1. `query_id (uint64)` - arbitrary request number
- 2. `amount (VarUInteger 16)` - jetton amount to burn
- 3. `response_destination (MsgAddress)` - where to send operation excess
- 4. `custom_payload (Maybe ^Cell)` - optional custom data

The implementation performs several critical validations: it verifies the sender is the wallet owner, checks for sufficient balance, and ensures adequate TON for transaction processing. After reducing the wallet balance, the contract sends a burn notification to the jetton master contract to decrease the total supply accordingly.

Compliance Note

The implementation precisely follows [TEP-0074](#) requirements for jetton supply reduction. The master contract authorization check prevents unauthorized supply modifications. This operation enables jetton supply management while preventing unauthorized jetton supply reduction.

Internal Transfer Implementation

Category	Severity	Location	Status
Compliance	Informational	ft/jetton-wallet.fc	Verified

Description

The `internal_transfer#178d4519` operation implements the recipient-side jetton transfer mechanism defined in [TEP-0074](#). This operation processes incoming jetton transfers, updates the recipient’s balance, and optionally notifies the owner.

Implementation Details

The implementation follows the exact TL-B schema from the standard:

```
internal_transfer#178d4519 query_id:uint64 amount:(VarUInteger 16)
                             from:MsgAddress
                             response_address:MsgAddress
                             forward_ton_amount:(VarUInteger 16)
                             forward_payload:(Either Cell ^Cell)
                             = InternalMsgBody;
```

The `internal_transfer` message includes:

- 1. `query_id (uint64)` - same as the original transfer query_id
- 2. `amount (VarUInteger 16)` - jetton amount being received
- 3. `from (MsgAddress)` - address of the sender who initiated the transfer
- 4. `response_address (MsgAddress)` - address to receive excess
- 5. `forward_ton_amount (VarUInteger 16)` - amount of TON to forward to the owner with notification
- 6. `forward_payload (Either Cell ^Cell)` - data to be passed to the owner in the notification

The implementation validates that the sender is either the master contract or another valid jetton wallet. It increases the balance by the specified amount, calculates storage fees, and processes forward requests when applicable. If `forward_ton_amount` is positive, it sends a notification to the wallet owner.

Compliance Note

The implementation fulfills [TEP-0074](#) requirements for receiving jettons. The strict source validation (only accepting transfers from the master or valid wallets) prevents unauthorized balance increases. The optional owner notification mechanism enables recipient awareness of incoming transfers. This operation completes the transfer process started by the sender's wallet, maintaining consistency between sender and receiver records.

Transfer Notification Implementation

Category	Severity	Location	Status
Compliance	Informational	ft/jetton-wallet.fc	Verified

Description

The `transfer_notification#7362d09c` implements the recipient notification mechanism defined in [TEP-0074](#). This message informs jetton recipients about incoming transfers when `forward_ton_amount` is non-zero.

Implementation Details

The implementation follows the exact TL-B schema from the standard:

```
transfer_notification#7362d09c query_id:uint64 amount:(VarUInteger 16)
                               sender:MsgAddress
                               forward_payload:(Either Cell ^Cell)
                               = InternalMsgBody;
```

The `transfer_notification` includes:

- 1. `query_id (uint64)` - same as the original transfer `query_id`
- 2. `amount (VarUInteger 16)` - amount of transferred jettons
- 3. `sender (MsgAddress)` - address of the previous owner of transferred jettons
- 4. `forward_payload (Either Cell ^Cell)` - data sent in the original transfer

The implementation generates this notification during `internal_transfer` processing when `forward_ton_amount` is greater than zero. The message is sent to the wallet owner’s address with precisely the `forward_ton_amount` of TON attached, using message mode 1 (pay transfer fees separately).

Compliance Note

The implementation fulfills [TEP-0074](#) requirements for `transfer_notification`. The notification allows recipient smart contracts to react to incoming jettons programmatically. This notification mechanism bridges the gap between jetton transfers and business logic on the recipient side.

Excesses Message Implementation

Category	Severity	Location	Status
Compliance	Informational	ft/jetton-wallet.fc, ft/jetton-minter-discoverable.fc	Verified

Description

The `excesses#d53276db` message implements the operation completion feedback mechanism defined in [TEP-0074](#). This message returns unused TON coins to the specified response address after completing jetton operations.

Implementation Details

The implementation follows the exact TL-B schema from the standard:

```
excesses#d53276db query_id:uint64 = InternalMsgBody;
```

The `excesses` message includes:

- 1. `query_id (uint64)` - same as the original operation `query_id`

The implementation sends this message after completing operations like `transfer` and `burn`. Any excess TON remaining after processing fees and forwarding requirements is returned to the `response_destination` address specified in the original operation.

Compliance Note

The implementation conforms to the [TEP-0074](#) requirements for excess TON handling. The `excesses` message ensures users receive unused transaction funds, improving economic efficiency.

Get-Methods Implementation

Category	Severity	Location	Status
Compliance	Informational	ft/jetton-wallet.fc, ft/jetton-minter-discoverable.fc	Verified

Description

The get-methods implement the data retrieval interfaces defined in [TEP-0074](#). These methods provide access to wallet balances, ownership information, and jetton metadata.

Implementation Details

The jetton wallet implements the required `get_wallet_data()` method with the signature:

```
(int, slice, slice, cell) get_wallet_data()
```

The `get_wallet_data()` method returns:

- 1. `balance (int)` - jetton amount owned by this wallet
- 2. `owner_address (slice)` - address of the wallet owner
- 3. `jetton_master_address (slice)` - address of the jetton master contract
- 4. `jetton_wallet_code (cell)` - code of the wallet contract

The jetton master implements two required methods:

```
(int, int, slice, cell, cell) get_jetton_data()  
(slice) get_wallet_address(slice owner_address)
```

The `get_jetton_data()` method returns:

- 1. `total_supply (int)` - total amount of issued jettons
- 2. `mintable (int)` - flag indicating if more jettons can be minted (-1 means mintable)
- 3. `admin_address (slice)` - address of the contract that controls the jetton
- 4. `jetton_content (cell)` - jetton metadata following TEP-0064 standard

5. `jetton_wallet_code (cell)` - code of the wallet contract

The `get_wallet_address(slice owner_address)` method accepts:

1. `owner_address (slice)` - address of the wallet owner

And returns:

1. `jetton_wallet_address (slice)` - the corresponding jetton wallet address for the owner

Compliance Note

The implementation fulfills all [TEP-0074](#) requirements for data retrieval interfaces. The get-methods provide essential chain analysis capabilities. The wallet data method enables balance verification and ownership confirmation. The master methods support supply verification and wallet address discovery.

Bounce Handling Implementation

Category	Severity	Location	Status
Compliance	Informational	ft/jetton-wallet.fc	Verified

Description

The bounce handling mechanism implements error recovery for failed jetton operations. While not explicitly defined in [TEP-0074](#) message formats, this functionality preserves jettons during failed [transfer](#) or [burn](#).

Implementation Details

The implementation processes bounced messages in the [on_bounce\(\)](#) function by first skipping the [0xFFFFFFFF](#) prefix, then extracting the operation code and parameters from the bounced message body:

```
() on_bounce (slice in_msg_body) impure {
  in_msg_body~skip_bits(32); ;; 0xFFFFFFFF
  (int balance, slice owner_address, slice jetton_master_address, cell
jetton_wallet_code) = load_data();
  int op = in_msg_body~load_uint(32);
  throw_unless(709, (op == op::internal_transfer()) | (op ==
op::burn_notification()));
  int query_id = in_msg_body~load_uint(64);
  int jetton_amount = in_msg_body~load_coins();
  balance += jetton_amount;
  save_data(balance, owner_address, jetton_master_address, jetton_wallet_code);
}
```

The [bounced](#) message structure being processed includes:

1. [op \(uint32\)](#) - operation code of the failed message ([internal_transfer](#) or [burn_notification](#))
2. [query_id \(uint64\)](#) - same as the original operation query_id
3. [jetton_amount \(VarUInteger 16\)](#) - jetton amount that were being transferred or burned

The function validates that the bounced message was either an [internal_transfer](#) or [burn_notification](#) operation. It extracts the jetton amount from the bounced message and adds it back to the wallet’s balance, effectively reverting the previously deducted jettons.

Compliance Note

The bounce handling implementation provides crucial jetton safety. Without this mechanism, jettons could permanently disappear when destination contract reject `internal_transfer` or when `burn_notification` fail. This safeguard ensures the jetton supply remains consistent even during operational failures, protecting users from losing jettons due to network or contract errors.

Discovery Functions Implementation

Category	Severity	Location	Status
Compliance	Informational	ft/jetton-minter-discoverable.fc, ft/discovery-params.fc	Verified

Description

The discovery functions implement the wallet address resolution mechanism defined in [TEP-0089](#). This extension to the base [TEP-0074](#) standard enables on-chain discovery of wallet addresses for specific owner addresses.

Implementation Details

The implementation follows the exact TL-B schema from the [TEP-0089](#) standard:

```
provide_wallet_address#2c76b973 query_id:uint64 owner_address:MsgAddress
                                include_address:Bool
                                = InternalMsgBody;

take_wallet_address#d1735400 query_id:uint64 wallet_address:MsgAddress
                              owner_address:(Maybe ^MsgAddress)
                              = InternalMsgBody;
```

The `provide_wallet_address` operation correctly handles all required parameters:

- 1. `query_id (uint64)` - arbitrary request number
- 2. `owner_address (MsgAddress)` - address for which to discover the wallet
- 3. `include_address (Bool)` - flag indicating whether to include owner_address in the response

The `take_wallet_address` response includes:

- 1. `query_id (uint64)` - same as the original operation query_id
- 2. `wallet_address (MsgAddress)` - the discovered wallet address (or addr_none if not resolvable)
- 3. `owner_address (Maybe ^MsgAddress)` - optional owner address, included if requested

The contract properly validates that the message value is sufficient and correctly calculates the wallet address using the standard wallet derivation algorithm. When a request is received, the contract formulates the appropriate response and sends it back with mode 64 (return remaining value) as required by the standard. If the address cannot be resolved (workchain mismatch), the contract correctly responds with `addr_none` for the `wallet_address` field. If the address is not `addr_std`, the contract will throw a deserialization exception.

Compliance Note

The implementation fully conforms to [TEP-0089](#) requirements. The discovery mechanism solves the problem of determining valid wallet addresses for any owner without relying on off-chain calculations.

Burn Notification Processing

Category	Severity	Location	Status
Compliance	Informational	ft/jetton-minter-discoverable.fc	Verified

Description

The `burn_notification#7bdd97de` processing implements the total supply reduction mechanism defined in [TEP-0074](#). This operation finalizes jetton burning by updating the master contract’s total supply after a wallet burns jettons.

Implementation Details

The implementation follows the exact TL-B schema from the standard:

```
burn_notification#7bdd97de query_id:uint64 amount:(VarUInteger 16)
                             sender:MsgAddress response_destination:MsgAddress
                             = InternalMsgBody;
```

The `burn_notification` message includes:

- 1. `query_id (uint64)` - same as the original burn `query_id`
- 2. `amount (VarUInteger 16)` - jetton amount being burned
- 3. `sender (MsgAddress)` - address of the jetton owner who initiated the burn
- 4. `response_destination (MsgAddress)` - address to receive excess TON

The implementation validates that the message came from a valid wallet. Upon validation, the master contract reduces the total supply by the burn amount and sends an excesses message to the `response_destination`.

Compliance Note

The implementation adheres to [TEP-0074](#) requirements for burn finalization. The sender validation prevents unauthorized reduction of total supply. The burn process ensures that the burned jetton amount is subtracted from the total supply.

Architectural Compliance

Category	Severity	Location	Status
Compliance	Informational	All files	Verified

Description

The implementation follows the distributed architecture described in [TEP-0074](#). This design separates jetton accounting into centralized supply management and distributed ownership records.

Implementation Details

The implementation establishes two primary contract types:

- 1. Jetton Master Contract (jetton-minter-discoverable.fc):
- 2. Jetton Wallet Contracts (jetton-wallet.fc):

Supporting utility files (jetton-utils.fc, discovery-params.fc, op-codes.fc) provide address calculation, operation codes, and helper functions for the primary contracts.

The implementation uses standard TON patterns for contract interaction, including state initialization calculation, contract address derivation, and message-based communication.

Compliance Note

The implementation fulfills the architectural requirements in [TEP-0074](#) "Rationale and alternatives" section. The distributed approach of "one wallet - one contract" provides security isolation between jetton owners and scalability for large numbers of jetton holders. The standardized communication protocols between wallets and the master maintain consistency across the entire jetton system.

06 Security Analysis

Our security assessment found that the implementation demonstrates a high level of security awareness and follows TON smart contract best practices. Key security aspects examined include:

1. Access control mechanisms properly restrict administrative functions and validate sender ownership
2. Economic security measures prevent overflow/underflow issues and include sufficient gas checks
3. Message handling correctly processes incoming messages, including bounced messages
4. Smart contract patterns are implemented properly, with no reentrancy vulnerabilities or unexpected state modifications

The distributed architecture of "one wallet - one contract" provides a robust approach to handling jetton ownership, which limits the impact of potential vulnerabilities to individual wallets rather than affecting the entire jetton supply.

07 Issues

During our audit of the TON Blockchain Jetton implementation, we identified the following findings.

Inability to Withdraw TON from Wallets After Admin Revocation

Category	Severity	Location	Status
Design limitation	Informational	ft/jetton-wallet.fc, ft/jetton-minter-discoverable.fc	Resolved

Description

The jetton implementation does not provide a mechanism to withdraw native TON from jetton wallet or master contracts if admin rights are revoked or lost. Once the `admin_address` is changed to an inaccessible address, any TON balance trapped in these contracts becomes permanently locked.

Impact

This design limitation can lead to permanent loss of TON funds that accumulate in the contracts. While this does not affect the jetton functionality itself, it represents poor economic design that can result in permanently locked funds in the TON ecosystem.

Recommendation

Implement a `withdraw_tons` operation in both jetton wallet and master contracts that allows the owner/admin to extract excess TON. For the wallet contract, this function should be restricted to the wallet owner. For the master contract, this should be restricted to the admin. This would prevent TON from being permanently locked if admin rights are transferred or lost. Consider adding a maintainer role that could withdraw TON but would not have minting privileges.

Inability to Withdraw Non-Jetton Assets

Category	Severity	Location	Status
Design limitation	Informational	ft/jetton-wallet.fc, ft/jetton-minter-discoverable.fc	Resolved

Description

The jetton wallet and master contracts lack functionality to extract non-native assets that might be received by these contracts. If other jettons or NFTs are sent to these contracts, they become permanently locked as there is no extraction mechanism implemented.

Impact

Any non-native assets accidentally sent to jetton wallet or master contracts are permanently lost. This could result in significant financial losses if valuable assets are mistakenly transferred to these contracts.

Recommendation

Implement asset recovery functions that allow the owner/admin to extract foreign assets from the contracts. These functions should support withdrawing other jetton types and NFTs that may have been mistakenly sent to the contract addresses. Consider adding a maintainer role that could withdraw assets but would not have minting privileges.

Insufficient Forward Amount for Transfer Notifications

Category	Severity	Location	Status
Integration risk	Informational	ft/jetton-wallet.fc	Resolved

Description

When integrating with the jetton standard, contracts may fail to process transfer notifications if the `forward_ton_amount` specified is insufficient to cover the processing costs. The standard does not enforce minimum values for `forward_ton_amount`, leaving it to the sender's discretion.

In the `jetton-wallet.fc` implementation, the `forward_ton_amount` is passed directly to the notification message without additional verification that it's sufficient for processing:

```
var msg_body = begin_cell()
    .store_uint(op::transfer_notification(), 32)
    .store_uint(query_id, 64)
    .store_coins(jetton_amount)
    .store_slice(from_address)
    .store_slice(either_forward_payload)
    .end_cell();

var msg = begin_cell()
    .store_uint(0x10, 6)
    .store_slice(owner_address)
    .store_coins(forward_ton_amount)
    .store_uint(1, 1 + 4 + 4 + 64 + 32 + 1 + 1)
    .store_ref(msg_body);
```

Impact

Contracts receiving jettons may fail to process the `transfer_notification` if the `forward_ton_amount` is insufficient. This can lead to inconsistent state in receiving contracts, and failed business logic execution. Contracts expecting to be notified about jetton receipts may function incorrectly if notifications cannot be processed.

Recommendation

While this is primarily an integration consideration, the standard implementation could be improved by:

1. Implement emergency handlers that can be used return lost funds

2. Adding documentation that emphasizes the importance of adequate `forward_ton_amount` and providing specifications about its recommended size

Implementers integrating with the jetton standard should be explicitly advised to provide sufficient `forward_ton_amount` for reliable notification processing.

No Automatic Return of Funds for Failed Notifications

Category	Severity	Location	Status
Integration risk	Informational	ft/jetton-wallet.fc	Resolved

Description

The jetton wallet implementation does not automatically return funds to the sender if `transfer_notification` message bounces. When a notification is sent as part of a `transfer` operation and the receiving contract fails to process it, the jetton transfer succeeds but the notification with any associated logic execution fails.

Impact

Contracts that send jettons and expect the recipient to execute specific logic upon receipt cannot guarantee that the logic will be executed. This behavior requires contract developers to implement emergency handlers that can be used return lost funds.

Recommendation

This is a documented behavior of the standard rather than a bug, but integrators should be clearly warned about this behavior. Contract developers who integrate with the jetton standard should:

1. Implement emergency handlers that can be used return lost funds
2. Be prepared to handle cases where notifications fail but the transfer itself succeeds

Inconsistent Total Supply After Failed Mint Operations

Category	Severity	Location	Status
Logic issue	Informational	ft/jetton-minter-discoverable.fc	Resolved

Description

The standard does not explicitly define the minting process, and the current implementation has a minor flaw. When a mint operation increases the `total_supply` but then fails during the `internal_transfer` processing at the jetton wallet, the `total_supply` remains increased even though the tokens were not actually created.

If the `mint_tokens` operation fails on the jetton wallet, there is no mechanism to roll back the `total_supply` increase.

Impact

This can lead to discrepancies between the reported `total_supply` and the actual sum of all wallet balances. Over time, with multiple failed mint operations, this inconsistency can grow, leading to inaccurate reporting and potential economic issues in systems that rely on the total supply value.

Recommendation

1. Document this limitation clearly so users and integrators are aware
2. Do not rely on the `total_supply` as an accurate source of truth about total wallet balances. Instead, if your service needs this value for business logic, you should calculate it by parsing all active wallets.

Potential Total Supply Inaccuracy

Category	Severity	Location	Status
Design limitation	Informational	ft/jetton-minter-discoverable.fc	Resolved

Description

The `total_supply` maintained by the jetton master contract may become inaccurate due to several edge cases not addressed in the implementation:

- 1. Tokens sent to zero address or otherwise inaccessible addresses, effectively burning them without updating `total_supply`
- 2. Accumulated discrepancies from mint operation failures

The master contract only updates `total_supply` in response to explicit `mint` and `burn_notification` operations.

Impact

The reported `total_supply` may not reflect the actual circulating supply of tokens. This can affect economic models, governance systems, or applications that rely on an accurate total supply figure.

Recommendation

- 1. Document this limitation clearly so users and integrators are aware
- 2. Do not rely on the `total_supply` as an accurate source of truth about total wallet balances. Instead, if your service needs this value for business logic, you should calculate it by parsing all active wallets.

Trapped TON Due to Gas Management Inefficiencies

Category	Severity	Location	Status
Gas optimization	Informational	ft/jetton-wallet.fc	Resolved

Description

The jetton wallet implementation has a gas management limitation that traps small amounts of TON in wallet contracts. The code calculates gas needs excessively to ensure operations complete successfully, but this approach leaves surplus TON that cannot be recovered.

Our security analysis confirms there is no mechanism to withdraw these trapped funds. Testing shows approximately 0.0035 TON remains after each transaction, accumulating to around 3,500 TON per million operations.

This represents a design limitation rather than a bug, but creates an economic inefficiency that becomes significant at scale.

Impact

TON accumulates in wallet contracts without a withdrawal mechanism. At scale, this becomes economically significant, effectively locking TON that could otherwise circulate in the ecosystem.

Recommendation

1. Implement a `withdraw_tons` function that allows wallet owners to extract excess TON
2. Consider more precise gas calculations based on operation size and current gas prices

08 Conclusion

The TON Blockchain Jetton implementation demonstrates full compliance with the [TEP-0074](#) standard. The code is mature, well-structured, and also implements the [TEP-0089](#) standard which extends [TEP-0074](#) with advanced functionality for Jetton discovery.

From a security perspective, the implementation correctly follows all requirements specified in both standards. The [TEP-0074](#) provides the core Jetton functionality while [TEP-0089](#) enhances it with standardized discovery mechanisms. The codebase implements necessary security measures, handles most edge cases properly, and applies appropriate access control across all operations in both standard implementations.